

Confidential
Disclosure: Email Recipients Only

Tech Lead Assignment

Scalable Notification Delivery Platform with Complex Event Processing.

72 Hours Only

Objective:	3
Functional Requirements:.....	3
Send Notification.....	3
Queue & Worker	3
Status Tracking.....	3
Scheduling Support	4
CEP Functional Requirements:.....	4
Tech Guidelines:	4
CEP Implementation Guidelines:	5
Deliverables:	5
Evaluation Criteria:.....	5
System Design & Architecture – 25%.....	5
2. Complex Event Processing (CEP) – 25%	6
3. Code Quality & Best Practices – 15%	6
4. Retry & Resilience Logic – 10%.....	6
5. Test Coverage – 10%.....	7
6. Documentation – 10%	7
7. Bonus (Optional) – 5%	7

Objective:

You want to introduce **real-time event processing** to dynamically control how notifications are routed based on rules or patterns. Design and implement a **Notification Delivery Platform** capable of receiving requests to send notifications (Email, SMS, Push), process them asynchronously, and track their delivery status. Emphasis is on **architecture, scalability, extensibility, and leadership thinking**.

Functional Requirements:

Send Notification

- a. REST API: POST /notifications
- b. Accepts payload like:

```
{  
  "user_id": "12345",  
  "country": "PHILIPPINES",  
  "channel": "EMAIL",  
  "message": "Your OTP is 9876",  
  "schedule_at": "2025-07-24T18:00:00Z"  
}
```

Queue & Worker

- c. Notifications should be queued and processed asynchronously.
- d. Simulate or actually integrate with Email/SMS gateways.
- e. Add retry mechanism for transient failures.

Status Tracking

- f. Persist and expose statuses: QUEUED, SENT, FAILED, RETRYING
- g. API: GET /notifications/:id/status

Scheduling Support

- h. Notifications can be scheduled for future deliveries.

CEP Functional Requirements:

i. Rule-Based Filtering:

- i. Only send notifications to users who have not received more than 3 messages in the last hour (rate limiting).
- ii. Example Rule: "count(channel == 'SMS', window: 1h) > 3 => DROP"

j. Dynamic Routing Based on Event Context:

- i. If the user's location is "PHILIPPINES" and it's between 10PM–6AM, delay notification until morning.
- ii. Example Rule: "user.country == 'PHILIPPINES' && now().hour < 6 => DELAY UNTIL 06:00"

k. Deduplication:

- i. Suppress identical messages sent to the same user within 5 minutes.
- ii. Example Rule: "dup(message, window: 5m) => DROP"

l. Enrichment Support (Optional):

- i. Enrich notification event with user profile (from mock service or cache) before evaluating rules.

m. Extensibility:

- i. Rules should be **externalized/configurable** not hardcoded.
- ii. Design should allow plug-in of more CEP rules or use a basic in-house engine.

n. Multi-channel

- i. Support EMAIL and SMS channels.
- ii. Design should be **extensible** for future PUSH, SLACK, etc.

Tech Guidelines:

- **Backend Language:** Java,
- **Framework:** Spring Boot
- **Queue:** In-memory queue is fine, or RabbitMQ.
- **DB:** PostgreSQL or SQLite/H2.
- **Email/SMS:** Use mock services (no need for real API keys).

- **Scheduling:** Can use simple in-memory delayed job or cron-like mechanism.

CEP Implementation Guidelines:

- You can:
 - Simulate a **lightweight CEP engine** (in-memory rule evaluation using Java)
 - OR use existing tools like **Esper**, **Drools**, or a simplified **custom evaluator**
- Keep **state per user/channel** for rate limiting or deduplication
- Store window state in-memory or simulate a store (e.g., RocksDB, Redis, or map)

Deliverables:

- Working Codebase (API, queue, processor, DB logic)
- **Architecture Document**
 - Key components (with diagrams preferred)
 - Queue vs synchronous decision
 - Scaling plan (what happens at 10K TPS?)
 - Failure scenarios and how handled
 - Justify your tech choices
- Test Cases
 - Unit tests for critical parts (channel handlers, retry logic)
 - If possible, add an integration test
- Setup Instructions
 - README with setup guide
 - Docker-compose (optional)
- A **CEP Engine or Rule Evaluator** component
- A rules-config.json or similar rule definition
- Unit tests for 2–3 rules
- Documentation section: “**How CEP works in this system**”

Evaluation Criteria:

System Design & Architecture – 25%

- **Scalability and throughput (8%)**
Can handle 1000+ notifications per second.

- **Modularity / Separation of concerns (6%)**
Clean separation between API, queue, worker, and delivery channels.
- **Fault-tolerance & Resiliency (5%)**
Handles transient errors, retries, and dead-letter scenarios effectively.
- **Extensibility (6%)**
Easy to add new delivery channels or CEP rules.

2. Complex Event Processing (CEP) – 25%

- **Rule-based evaluation logic (8%)**
Implements time-based, windowed, or content-based rules accurately.
- **Configurability of rules (6%)**
Rules are externalized via JSON/YAML or DSL, easy to maintain.
- **Rule engine design (5%)**
Reusable, extensible, and cleanly integrated with the platform.
- **State management for rules (6%)**
Correctly handles deduplication, rate-limiting, and window state.

3. Code Quality & Best Practices – 15%

- **Clean, readable code (5%)**
Consistent naming, formatting, no dead code, well-commented.
- **SOLID principles (5%)**
Adheres to object-oriented design principles and separation of concerns.
- **Reusability and DRY adherence (3%)**
Avoids duplication, encourages reusable logic.
- **Dependency injection and configs (2%)**
Uses DI, environment-specific config files, avoids hardcoded values.

4. Retry & Resilience Logic – 10%

- **Retry logic correctness (4%)**
Uses max retries, exponential backoff, and handles retryable exceptions.

- **Dead-letter handling (3%)**
Implements dead-letter logic for unrecoverable failures (optional).
- **Resilience to external failures (3%)**
Degrades gracefully during SMS/Email provider failures.

5. Test Coverage – 10%

- **Unit tests (4%)**
Covers core logic: retry, rule matching, scheduling.
- **Integration tests (3%)**
Validates full flow across modules.
- **Edge case handling (3%)**
Tests invalid inputs, time delays, and duplicate scenarios.

6. Documentation – 10%

- **Architecture document (4%)**
Explains system components, includes diagrams and flow.
- **Setup instructions (README) (3%)**
Includes local setup, build/run/test steps.
- **Assumptions and trade-offs (3%)**
Clearly outlines what's mocked, skipped, or assumed.

7. Bonus (Optional) – 5%

- **Observability (logs, metrics) (2%)**
Uses structured logging, Prometheus, or tracing.
- **Docker-compose for local setup (1%)**
Includes Docker setup for DB, app, mock queues.
- **Swagger/OpenAPI docs (1%)**
Includes API documentation.
- **Async scheduling / cron (1%)**
Implements actual delay queue or cron scheduling.